

流式翻译管道

按需实时翻译，由播放触发。当用户按下播放键收听未翻译的有声书时，系统会将 **30 秒分段**分发给 GPU 工作节点，缓冲三分钟的翻译音频，然后开始播放。已预翻译的书籍从缓存中即时加载。（整章处理属于**批量**管道；流式管

参考部署中未启用

本管道需要一个本项目并不提供的 STT 后端。维护者的 RunPod 账户已**停用**， Vast.ai 从未启用过，因此没有配置任何 STT 端点。library/localization/pipeline.py在被要求转录时会抛出 RuntimeError("No STT provider configured")。

下文描述的是代码的实际写法，代码本身未变。但在**当前部署下，不会分发任何任务，也不会完成任何工作**。三个 systemd 单元处于启用并运行状态；试听层每 5 分钟如实记录一次：

```
SAMPLER STALE: no sampler_job has completed in N days (threshold 3).
The timer is running; the work is not.
```

已缓存的章节仍可正常播放 —— 缓存不受影响。如需关闭该子系统，请参见 docs/MULTI-LANGUAGE-SETUP.md 中的” Turning audio translation off” （注意 upgrade.sh 每次运行都会重新启用这些单元，因此仅 systemctl disable 并不持久）。

为什么需要流式翻译

预先批量翻译整个音频库（每个章节的 STT + DeepL + TTS，每种语言）将花费数百 美元的 GPU 费用，因此本管道的设计目标是：只为用户实际收听的内容付费。

（早期版本引用了 v8.3.0 时的数字：“1,861 本有声书 / 已预翻译 327 本 / 5,245 个章节 / 剩余 1,534 本”。那是单一部署在某一时间点的数据，此后已经漂移， 因此不再维护而予以删除。另请注意，现存 chapter_subtitles 记录中有相当一部分 标记为 stt_provider='vastai-whisper' —— 这是某个已在 v8.3.10.6 中移除其客户端 代码的服务商留下的历史输出。）

流式翻译的解决方案是：只为用户实际收听的内容付费。

两条管道，一个缓存

管道	触发方式	处理方式	输出
批量 (batch-translate.py)	操作员手动运行 + 队列	整章处理，后台运行	永久 VTT + TTS 音频
流式 (streaming_translate.py)	播放触发	30 秒分段，实时处理	分段 → 合并 VTT

两条管道都写入同一个永久缓存（chapter_subtitles和chapter_translations_audio表）。一旦某个章节被任一管道翻译，用户的收听习惯会逐渐填充缓存，其余部分由批量管道补齐。

关于该设计有两点需要说明。scripts/batch-translate.py **仅由操作员手动运行** —— 没有任何 systemd 单元或定时器调用它，因此” 批量管道在空闲时处理剩余内容” 描述的是一种意图，而非既有机制。而在参考部署中，由于 STT 后端，两条管道都不运行。

端到端播放流程

第一阶段 — 打开应用（GPU 预热）

当应用打开且用户语言不是英语时，前端发送 `POST /api/translate/warmup` 请求。这会向 `streaming_sessions` 写入一行 (`audiobook_id=0`, `locale='warmup'`)。

目前没有任何代码消费这一行。在 `scripts/stream-translate-worker.py` 中 `grep warmup` 没有任何匹配，处理函数本身也这样——“实际的 GPU 预热将由 翻译守护进程在看到该信号时处理” (`streaming_translate.py:1384-1404`)。而这样的守护进程并不存在。既不会发出预热请求，也不会验证连通性；该端点是一个空操作占位符。

端点的 `streaming/backlog` 划分请参见 `docs/SERVERLESS-OPS.md`；卡住领取的重置 约定（实时 60 秒、试听 2 小时）请参见 `docs/TRANSLATION-MONITOR.md`。本节的早期 版本提到过“预热过期（15 分钟）“与”卡住分段回收（10 分钟）“——代码中并不存在这两个计时器。

第二阶段 — 按下播放

`shell.js` 调用 `streamingTranslate.check(bookId, locale)`，向协调器发送 `POST /api/translate/stream`：

```
播放器 → 协调器 API → 数据库查询：
├── chapter_subtitles 存在？（批量缓存）
├── chapter_translations_audio 存在？（批量 TTS 缓存）
├── 都存在 → { state: "cached" } → 即时播放
└── 缺失 → { state: "buffering", session_id, segment_bitmap }
```

第三阶段 — 缓冲状态

前端状态机从 `IDLE` 转换到 `BUFFERING`：

1. **视觉覆盖层**从播放器栏上方滑出 — 金色主题进度条显示分段完成情况（例如“3 / 6”）
2. **本地化音频通知**通过预生成的 `edge-tts` 语音播放（中文：“请稍候，正在为您翻译本书。字幕和语音朗读即将开始。”）
3. **主音频暂停** — 等待期间无需播放英文旁白

协调器同时执行：

- 为**整个当前章节**创建 `streaming_segments` 行，以 **P0** 优先级入队
- 为**整个下一章节**创建行，以 **P1** 优先级（预取）入队
- 每行代表一个 30 秒分段：(`audiobook_id`, `chapter_index`, `segment_index`, `locale`, `state='pending'`)

两次调用都经由 `_ensure_chapter_segments(...)`，其约定为“以指定优先级确保整个章节的分段行存在” (`streaming_translate.py:476, 1062-1069`)。

6 个分段的光标窗口属于跳转（seek）逻辑，而非按下播放时的逻辑。 `BUFFER_AHEAD_SEGMENTS = 6`（约 3 分钟）仅在 `handle_seek_impl (:837-857)` 中生效。按下播放时整章都会入队，实际先翻译哪些取决于工作节点的领取顺序。

完整的**四层**语义请参见下文的“优先级模型（以光标为中心）”“章节。

第四阶段 — GPU 工作节点处理

`stream-translate-worker.py` 按优先级顺序轮询 `streaming_segments` 表并处理 每个分段：

1. 原子性地获取下一个待处理分段（按优先级、章节、分段排序）
2. ffmpeg 流复制 → 从章节中提取 30 秒音频片段
3. STT (GPU 上的 faster-whisper) → 原始英文转录
4. 翻译 (DeepL API) → 翻译文本
5. 生成带时间戳的 VTT
6. 根据分段在章节中的位置偏移时间戳
7. POST /api/translate/segment-complete → 报告内联 VTT 内容

P0 光标缓冲分段优先处理，以便播放尽快恢复。满足 3 分钟缓冲后，工作节点继续处理 P1（向前追赶）以保持领先于光标，P2（回填），把光标后方的时间线补齐，保证侧边字幕面板和将来向后拖动时的连贯性。

第五阶段 — 实时推送

当协调器收到分段完成回调时：

1. 在数据库中将分段状态更新为 completed
2. 通过 WebSocket 向所有连接的客户端广播 segment_ready
3. 广播 buffer_progress，包含已完成/总数计数

前端接收这些事件并实时更新进度条。

第六阶段 — 达到缓冲阈值

当 6 个分段完成（3 分钟音频）时，状态机从 BUFFERING 转换到 STREAMING：

- 覆盖层隐藏
- 通知音频停止
- 主音频**恢复** — 翻译字幕可用
- GPU 工作节点继续处理播放光标前方的剩余分段

第七阶段 — 跳转处理

操作	行为
缓冲范围内 ±30 秒	即时 — 分段已缓存，无中断
跳转超出缓存范围	POST /api/translate/seek → 根据新光标重新排优先级 → 重新进入缓冲
跳转到批量缓存的章节	即时 — 已在永久缓存中

跳转超出缓冲范围时：所有现有的待处理分段会被降级为 **P2**；新光标前方的 6 个分段被提升或新插入为 **P0**（光标缓冲填充）；缓冲之后直至章末的剩余部分排入 **P1**（向前追赶）；上一段已翻译尾部与新光标之间的空缺则排入 **P2**（回填），以保证侧边字幕面板和将来任何向后拖动操作的连贯性。

停止播放时：该（书籍, 语言）下所有 state='pending' 的行都会被 **DELETE 删除**（stop_streaming_impl, streaming_trans881）。已进入 processing 的分段不受影响 — 工作节点会处理完，segment-complete 回调照常返回。
v8.3.2 之前这里是把待处理行降级，但工作节点排空 p0/p1 后就会继续啃降级后的行，导致”停止”根本停不下来。恢复播

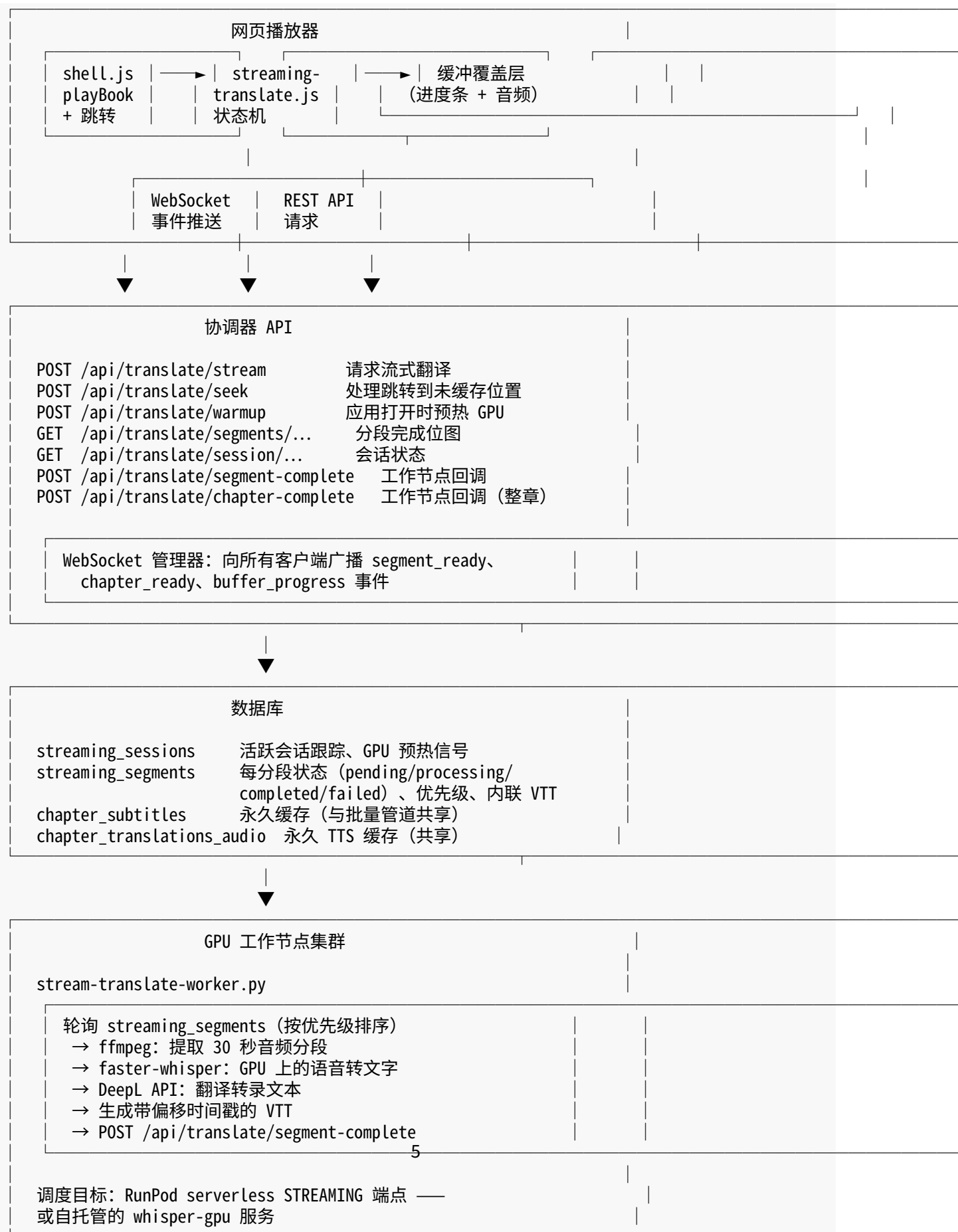
第八阶段 — 合并

当某章节的所有分段完成后，_consolidate_chapter() 执行：

1. 从所有分段行读取 VTT 内容
2. 去除重复的 WEBVTT 头部，合并为单个文件
3. 写入 subtitles/{audiobook_id}/ch{N}.{locale}.vtt

4. 插入 `chapter_subtitles` — 与批量管道使用的同一永久缓存合并后，该章节与批量翻译的章节无法区分。

架构图



设计参数

参数	值	原因
分段时长	30 秒	L40S 处理约 2-3 秒；足够小以实现低延迟
缓冲阈值	6 个分段（3 分钟）	足够的缓冲使播放不间断，同时 GPU 保持领先
P0 —— 光标缓冲填充	光标前方 6 个分段	播放恢复前必须就绪
P1 —— 向前追赶	从光标缓冲到章末 / 下一合理断点	播放过程中保持 GPU 领先于光标
P2 —— 回填	从上一段已翻译尾部到光标	避免字幕面板和向后拖动的连续性保障

优先级模型（以光标为中心，自 v8.3.8 起为四层）

调度器**以光标为中心**，而不是以章节为中心。分段根据与用户当前播放光标的关系被排入四个优先级之一。v8.3.8 新增 **p2 = 试听（sampler）** 独立层级，将原先的回填工作下移至 p3，从而确保 6 分钟预翻译试听永远不会抢占实时播放：

优先级（数值越小越紧急）：

- 0 P0 —— 光标缓冲填充。填充光标前方约 3 分钟（6 个分段），播放恢复前必须先到位。****仅限当前正在播放的书籍。****
- 1 P1 —— 向前追赶。在光标缓冲之后继续产出分段，直到章末 / 下一合理断点。仅当用户跳转或停止时才会被降级。****仅限当前正在播放的书籍。****
- 2 P2 —— 试听（自 v8.3.8 起）。每本书开头的 6 分钟预翻译，适用于每个启用的非英语语言，成本有限。详见 ``docs/SAMPLER.md``。****数据库触发器强制保障****：任何尝试以 `priority<2` 插入/更新 `origin='sampler'` 行的操作都会被引擎 ABORT。
- 3 P3 —— 回填与其他批量工作。产出上一段已翻译尾部与光标之间的分段；必须在以上优先级都满足后才运行，以保证侧边字幕面板和将来向后拖动时的上下文连贯。

根据触发器与不变量：当前正在播放书籍的实时播放（p0/p1）永远优先于任何其他书籍的试听工作。试听绝不会从正在收听的用户手中夺走 GPU 的处理槽位。

跳转超出缓冲范围时：现有待处理的实时分段降级为 ****P2****（不是 P3——``handle_seek_impl:838-843`` 写入的是 2）；新光标前方 6 个分段被提升或新插入为 P0；章末剩余部分排入 P1。

停止播放时：所有待处理的实时分段被 DELETE 删除，而非降级（见上文）。

工作节点的领取顺序为 `ORDER BY s.priority ASC, CASE WHEN sess.active_chapter IS NOT NULL AND s.chapter_index = sess.active_chapter THEN 0 ELSE 1 END, s.chapter_index ASC, s.segment_index ASC`（`scripts/stream-translate-worker.py:336-341`）—— 新增了“当前章节优先”的次级排序，使得播放器进入第 N+1 章后，该章的 P0 行能够胜出。优先级优先的整体形状未变；v8.3.8 扩展了优先级层级，为试听提供了独立的受保护槽位。

试听与本模型的互动

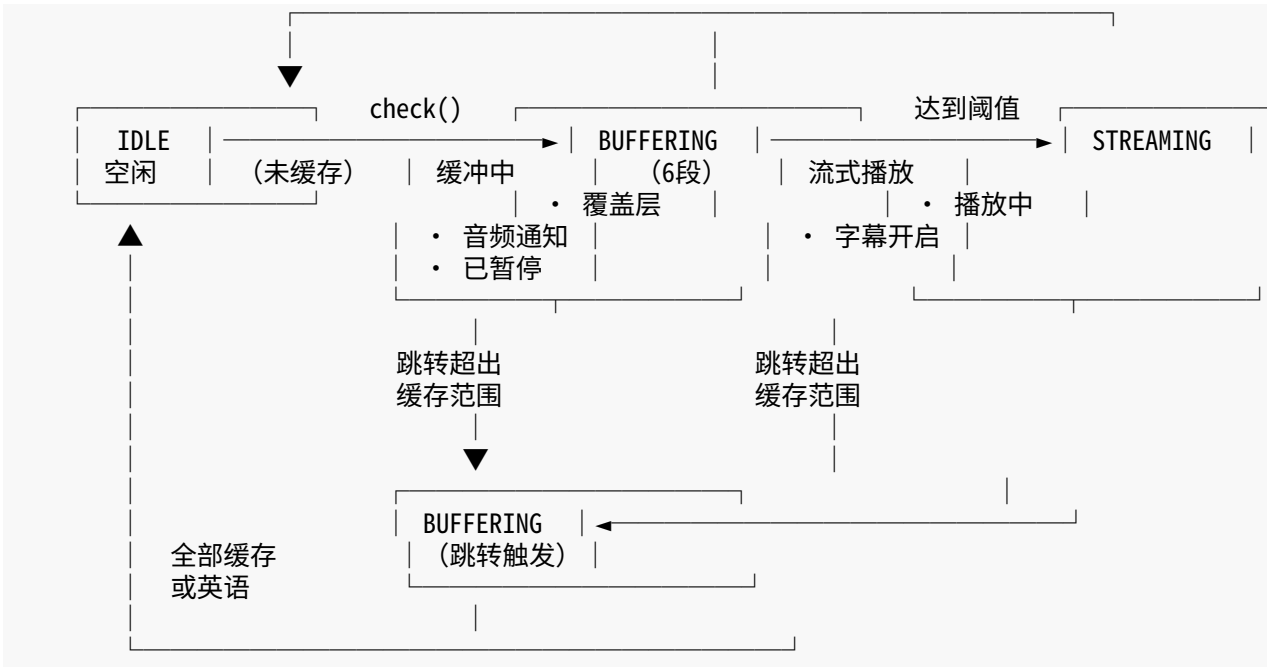
试听在图书入库时持续以 p2 运行，但绝不会与实时播放（p0/p1）竞争。当用户播放试听并越过**自适应缓冲触发阈值**（若所 STT 后端均无就绪工作者则为第 3 段；若任一后端温启则为第 4 段——详见 `docs/SAMPLER.md`）时，前端会调用

POST /api/translate/sampler/activate, 从光标向前创建 p0/p1 分段。GPU 冷启动的过程发生在用户仍在收听已缓存试听 6 分钟试听 结束时, 实时缓冲区已经开始填充。过渡无缝, 用户几乎不会看到缓冲圈。

状态转换总览

事件	P0 (光标缓冲)	P1 (向前追赶)	P2 (回填)
按下播放	光标前方 6 个分段	当前章节剩余部分	(空)
跳转超出缓冲圈	光标前方 6 个分段	缓冲之后的剩余	所有原待处理分段 + 上一段尾部到光标的空缺
停止	(空)	(空)	所有待处理分段
恢复播放	光标前方 6 个分段 (从 P2 重新提升)	章节剩余部分 (重新提升)	上一段尾部到光标的剩余

状态机



控制批量翻译

批量管道独立运行, 调度目标是 BACKLOG serverless 端点池 (冷池, min_workers=0)。调度由 API 内部触发, 也可通过 scripts/batch-translate.py 手动触发 —— 该脚本读取 translation_queue 并按章节逐一处理待处理行。

```
# 手动执行一次批量处理
sudo -u audiobooks /opt/audiobooks/library/venv/bin/python \
/opt/audiobooks/scripts/batch-translate.py
```

无需管理 GPU 生命周期 —— serverless 端点会自动缩容至零, 因此只需为实际 翻译的章节付费。BACKLOG 端点池在空闲时的成本为 0 美元。

卡死检测: streaming_segments 表中 processing 状态超过 10 分钟的分段, 会在流式工作节点的下一次轮询中被回收重试。API 的 reconcile 循环重置为 pending。

文件列表

文件	用途
library/backend/api_modular/streaming_translate.py	协调器 API (14 个端点)
library/web-v2/js/streaming-translate.js	前端状态机
library/web-v2/css/shell.css	缓冲覆盖层样式
library/web-v2/shell.html	覆盖层标记
scripts/stream-translate-worker.py	流式 GPU 工作节点 (分段处理, 对接 STREAMING 端点池)
scripts/stream-translate-daemon.sh	流式工作节点的长驻外壳脚本
scripts/batch-translate.py	批量工作节点 (章节处理, 对接 BACKLOG 端点池)
systemd/audiobook-stream-translate.service	流式工作节点服务单元
library/localization/pipeline.py	共享 STT → 翻译 → VTT 管道 (_remote_stt_candidates 按 STREAMING 或 BACKLOG 调度)
library/web-v2/audio/translation-buffering-*.mp3	本地化通知音频

数据库架构 (迁移 003)

```

CREATE TABLE streaming_sessions (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  audiobook_id INTEGER NOT NULL,
  locale      TEXT NOT NULL,
  state       TEXT DEFAULT 'buffering',    -- buffering, streaming, completed, warmup
  active_chapter INTEGER DEFAULT 0,
  buffer_threshold INTEGER DEFAULT 6,
  gpu_warm    INTEGER DEFAULT 0,
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE streaming_segments (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  audiobook_id INTEGER NOT NULL,
  chapter_index INTEGER NOT NULL,
  segment_index INTEGER NOT NULL,
  locale      TEXT NOT NULL,
  state       TEXT DEFAULT 'pending',    -- pending, processing, completed, failed
  priority    INTEGER NOT NULL DEFAULT 2, -- 0=P0 光标缓冲, 1=P1 向前追赶, 2=P2 试听/回填
  worker_id   TEXT,
  vtt_content TEXT,                      -- 目标语言 (已完成分段) 的内联 VTT
  source_vtt_content TEXT,              -- 源语言 (英文) VTT (v8.3.2+)
  audio_path  TEXT,                    -- 每个分段的 opus 路径, 由工作节点写入 (v8.3.2+)
  retry_count INTEGER DEFAULT 0,        -- 瞬时失败重试计数 (v8.3.2+)
  started_at  DATETIME,
  completed_at DATETIME,
  UNIQUE(audiobook_id, chapter_index, segment_index, locale)
);

```

该表结构通过多个数据迁移脚本逐步演进 —— 003_streaming_segments.sh (MIN_VERSION=8.3.0, 创建两张表)、005_streaming_retry_count.sh、006_streaming_source_vtt.sh、007_streaming_retry_count.sh, 以及 008_streaming_origin_and_sampler.sh (MIN_VERSION=8.3.2+)

origin 列、sampler_jobs 表和两个 RAISE(ABORT) 优先级触发器)。每个脚本均幂等 (PRAGMA table_info 守卫) 且按版本边界门控 (MIN_VERSION)，跨版本升级仅 填充缺失的部分。

上方的 SQL 是简化版 —— 省略了 origin、error、created_at、外键、三个索引、sampler_jobs 表以及优先级触发器。权威架构请以迁移脚本为准。

在途 VTT 拼接 (v8.3.7+)

清单和字幕获取路由将 chapter_subtitles (已完成、落盘的 VTT 文件) 与 streaming_segments 的实时索引合并，即便某章 VTT 尚未完成整合，只要第一个分段完成落库，就会立即出现在字幕列表中。

- `/api/audiobooks/<id>/subtitles` 返回两者的并集：(a) chapter_subtitles 中的缓存行，(b) 由 streaming_segments 按 (chapter_index, locale) 去重构建的索引。subtitles.js 的轮询无需等到章节整合就能发现正在流式处理的字幕轨。
- `/api/audiobooks/<id>/subtitle/<chapter>/<locale>` 在磁盘无缓存文件 (或 chapter_subtitles 有记录但文件缺失) 时，直接从 streaming_segments 构建拼接后的 VTT 返回。拼接过程剥离每个分段的 WEBVTT 头部，按 segment_index 顺序输出单一的 WEBVTT + 连续 cues。
- 对于 locale='en'，拼接器读取 source_vtt_content (Whisper 转录与 语言无关)；其他语言读取 streaming_segments.locale 匹配的 vtt_content。
- 拼接的 VTT **永不缓存到磁盘** —— 每次请求都从分段行重新构建，保证后到的分段在下一次拉取时即可显现。
- 错误辨别得到保留：chapter_subtitles 中存在行但磁盘文件缺失仍返回 VTT file missing on disk (404)；完全没有记录则返回 Subtitle not found (404)。

遗留队列的延迟状态 (v8.3.7+)

library/localization/queue.py::get_book_translation_status 将非英文 locale 的 pending / processing / failed 行折叠为 {"state": "deferred", "reason": "streaming_pipeline"}，避免将流式管道上线前遗留的批处理失败暴露给 UI。在此之前，首次打开任一尚未翻译的 zh-Hans 书都会显示来自 translation_queue 的过期字幕生成失败 — No STT provider configured 提示 —— 这些记录早在旧批处理 worker 停止运转数月前就已经是失败状态。非英文 locale 现在的规范进度展示面板是流式 overlay (实现在 library/web-v2/js/streaming-translate.js 内，标记位于 shell.html 的 #streaming-overlay —— 并不存在 streaming-overlay.js 这个文件)；已完成的遗留记录 (合法的磁盘 VTT 情况) 仍按原样通过。'en' locale 豁免 —— 英文的 STT 失败是真实情况，不是过期数据。

安全性

所有路由处理程序在边界处验证输入：

- **语言区域**：_sanitize_locale() 强制执行 `^[a-zA-Z]{2}(\?:-[a-zA-Z0-9]{1,8})?$` — 拒绝路径遍历 (../) 和日志注入
- **整数 ID**：audiobook_id、chapter_index、segment_index 在任何数据库查询或文件系统操作之前强制转换为 int
- **工作节点回调** (segment-complete、chapter-complete)：仅供 GPU 工作节点调用的内部端点，不暴露给浏览器客户端